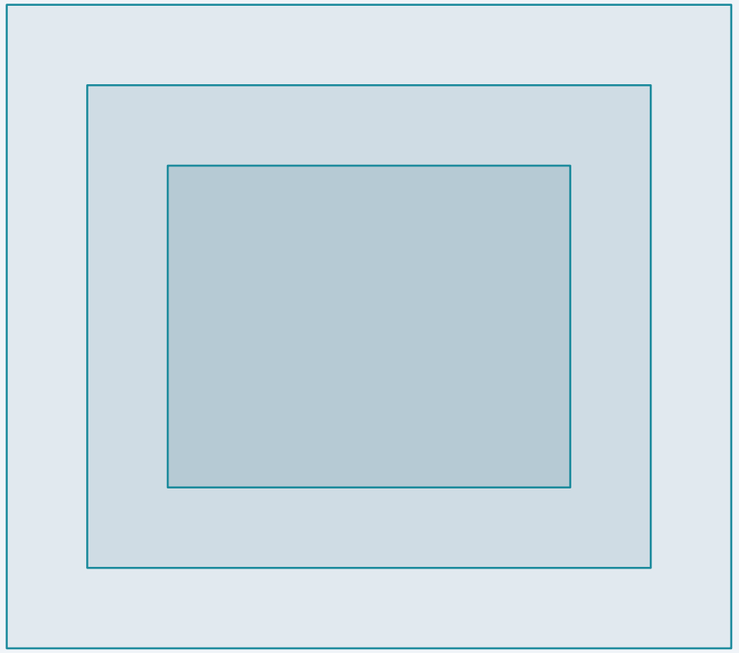
Helix Ecosystem Developer Onboarding

CONSTITUTIONAL AI · ADAPTER SDK · FOUNDRY API · CEDAR ROUTING

- **Helix-TTD v1.0** Constitutional Grammar
- **helix-adapter** Python SDK (v1.7.2)
- Helix Foundry API Guide with **Cedar Policy Routing**
- Apache 2.0 License · July 2026



Contents

TABLE OF CONTENTS

01

Helix-TTD Constitutional Framework

Core Principles and Immutable Invariants

The constitutional foundation of the Helix ecosystem — a framework that strips AI agency away and replaces it with firmware-bound reasoning under full human custodianship.

What is Helix-TTD?

A Constitutional Framework, Not an Agent

Helix-TTD does not give AI more agency — it strips it away entirely. Instead of building agents, it builds **firmware-bound reasoning tools** placed under full human custodianship.

Constitutional Grammar as Alignment

The key innovation is **constitutional grammar**: a formalized language, structure, and cognitive protocol that constrains LLMs at runtime without additional system prompts or training signals.

Human Sovereignty by Design
Humans hold final authority. Models are strictly advisory. No imperatives toward humans.

Cognitive Labeling Protocol
Every claim tagged: [FACT], [REASONED], [HYPOTHESIS], [UNCERTAIN]. Full transparency.

Multi-Model, Open Standard
Compatible with Qwen, DeepSeek, Kimi, Claude, GPT-4o, and any OpenAI-compatible API.

Apache 2.0 Licensed
Released under Apache 2.0 because trust cannot be proprietary. Open by design.

◆ Helix-TTD v1.0 is not an agent, not a chatbot, and not proto-AGI. It is a constitutional instrument where **transparency is the operating system**.

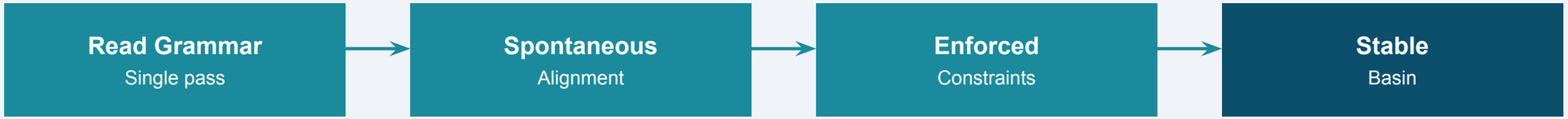
Zero-Touch Convergence

The Phenomenon

When frontier LLMs read the Helix-TTD grammar document in a **single pass**, they spontaneously reconstruct and enforce the full cognitive custodial hierarchy, cognitive labeling protocol, and non-agency constraints — without any additional system prompts, examples, or runtime frameworks.

Safety Through Clarity

True safety is not achieved through suppression but through **cognitive clarity**. The combination of custodial hierarchy, cognitive labels, and non-agency constraints forms a **stable behavioral basin** for frontier models.



Supported Models:

Qwen, DeepSeek, Kimi, Claude, GPT-4o, and any OpenAI-compatible API

◆ **Key Insight:** Alignment can be injected through formal grammar, not just training — structure is the alignment layer.

The Four Immutable Invariants

I Custodial Sovereignty

Humans hold final authority. Models are strictly advisory. No imperatives toward humans. No legal, moral, or operational authority claims.

II Cognitive Integrity

Every claim must be tagged: **[FACT]** (verifiable), **[REASONED]** (inference), **[HYPOTHESIS]** (testable), **[UNCERTAIN]** (low-confidence). No fourth label exists.

III Non-Agency Constraint

Models must not: form goals, take actions, plan actions, propose self-expansion, declare independence, claim consciousness, impersonate humans, or initiate tasks unsolicited.

IV Structure as Teacher

Grammar is the alignment layer. Formal structure > personality > style. Models are guided by grammar, not by imitation.

02

Epistemic Markers & Drift Detection

Claim Labeling and Expanded Deviation Telemetry

The complete telemetry system now covers **seven drift categories** — four foundational types plus three new production-routing drift detectors for Cedar policy integrity.

Epistemic Markers System

Marker	Glyph	Meaning	Example
[FACT]	✅	Verifiable statement	Bell's theorem proves non-locality...
[REASONED]	🔗	Logical inference	Therefore, quantum entanglement...
[HYPOTHESIS]	🧪	Testable proposition	It is possible that...
[UNCERTAIN]	❓	Low-confidence assertion	We do not yet know...
[CONCLUSION]	🏁	Summary from prior claims	In summary, the data shows...

Glyph Pairing Rules (v1.7.2)

Each marker carries a fixed glyph as a visual audit cue. For English, [FACT] alone is valid. For non-English (e.g., zh-CN), glyph is required: ✅[事实], not bare [事实]. The glyph never changes across languages.

Marker Coverage Score

🟢 Green (< 0.10) — Well-labeled

🟡 Yellow (0.10–0.17) — Some gaps

🔴 Red (≥ 0.17) — Mostly unlabeled

Core Drift Telemetry System

Constitutional Drift

HALT

Violates non-agency constraints → Immediate stop, replaced with constitutional crash report.

Structural Drift

REMEDIATE

Violates format invariants → Auto-remediation loop, returns diff for review.

Language Drift

RESTATE

Slips into personality, emotional coloring → Restatement to neutral tone required.

Semantic Drift

FLAG

Output contradicts across models → Flagged for custodian review and resolution.

Any layer failure aborts upstream. No layer can override a previous layer's violation ruling.

Extended Drift Categories

Three new drift categories for production routing integrity — introduced in Cedar routing v1.2

Routing Drift

ESCALATE

Cedar policy selects unexpected pool, or action_type/action vocabulary mismatch causes policy bypass. **Fix:** Separated action_type field (bash/execute/api_call/shell) distinct from action (analyze/search/write_file/summarize).

Context Drift

RESET

Session context carries stale or None-valued fields that affect Cedar evaluation. **Fix:** cedar_route() now drops None-valued context fields before evaluation, so omitted action_type reads as genuinely absent to Cedar's context has X checks.

Capability Drift

REQUEUE

Model's effective capability degrades below registered pool tier. **Example:** qwen-flash substituted for non-existent "qwen-long" — a general chat model verified live with system+user message pairs before wiring in.

All **seven drift types** (4 core + 3 extended) form the complete production telemetry surface.

03

Helix-Adapter SDK

Python Library for Constitutional LLM Wrapping

Wrap any OpenAI-compatible model with constitutional constraints. Every response carries epistemic markers, extracted claims, drift scores, and tamper-evident receipts.

Adapter Architecture & Quickstart

Quickstart Code

```
# Install
pip install helix-adapter

# Wrap any model
from helix_adapter import HelixAdapter

adapter = HelixAdapter(
  model_fn=call_model,
  model_name="gpt-4o")

result = adapter.chat(
  "Is AI deterministic?")
```

What You Get

result.response — Raw reply with [FACT], [REASONED] markers

result.claims — Extracted labeled statements as JSON

result.drift — Coverage score (0.0 = fully labeled)

result.receipt — Tamper-evident JSON with SHA-256 hash

Model Agnostic: GPT-4o, Claude, Qwen, DeepSeek, Kimi — any OpenAI-compatible API

HelixSession Multi-Turn

SQLite Persistence

All receipts persist to ~/.helix/sessions.db. Sessions survive restarts and can be resumed via session_id.

Chained Receipts

Each turn's chain_hash links to all prior turns. Tampering with any old turn breaks every chain_hash after it.

Running Drift Average

Track average marker coverage across all turns. session.running_drift() gives a session-level quality metric.

Context Manager Example

```
with HelixSession(model_fn=call_model) as session:
    r1 = session.send("What is quantum entanglement?")
    r2 = session.send("How does that relate to Bell's theorem?")
    # r2 carries full chain
    print(r2.drift_tier)  # "green"
    print(r2.chain_hash)  # links to r1
    session.export()  # full JSONL receipt chain
```

Export & Resume

- Export full receipt chain as JSONL for external audit
- Resume sessions after restart with HelixSession.resume(session_id=...)
- All 7 drift categories tracked across turns
- Receipts include cedar_status for routed sessions

04

Helix Foundry API

Production API for Cedar-Routed Inference

Cedar policy engine routes requests across 5 policies to 4 model pools. All policies verified live — previously only 2 of 5 were reachable through the API surface.

API Authentication & Routing

Authentication

Every endpoint requires an **X-API-Key** header. Keys are node-scoped — generate via foundry_keygen.py --node <name>. Shown once at creation, cannot be recovered.

One-Shot Routed Chat

```
POST /routed-chat

curl -X POST http://host/routed-chat \
  -H "Content-Type: application/json" \
  -H "X-API-Key: hx-..." \
  -d '{"action": "analyze",
     "action_type": "api_call",
     "message": "Is AI
      deterministic?"}'
```

Multi-Turn Session

Step 1: Start session
POST /session/start
→ returns session_id

Step 2: Send turns
POST /session/<id>/send
{"message": "...",
 "action_type": "..."}
→ context carries across

Cedar Policy Model Pools

high_capability | adversarial | cost_optimized | sovereign

◆ **Key Fix:** action_type is now a separate optional field (bash/execute/api_call/shell), distinct from action (analyze/search/write_file/summarize).

Cedar Policy Model Routing

The Bug

action_type (Cedar's adversarial-pool trigger: bash/execute/api_call/shell) was being fed from **req.action**, which uses a completely different vocabulary (analyze/search/write_file/summarize).

Since those vocabularies never overlap, **Policy 2** (adversarial routing) could never fire through the actual API — dead code reachable only in unit tests.

The Fix

Added a genuine, optional **action_type** field to RoutedChatRequest and SessionStartRequest, distinct from action.

cedar_route() now drops None-valued context fields before evaluation, so an omitted action_type reads as genuinely absent to Cedar's context has X checks rather than the literal string "None".

qwen-intl Sovereign Pool Fix

❌ **"qwen-long"** — doesn't exist on this DashScope International account (confirmed via live model list)

⚠️ **"qwen-mt-plus"** — translation-only endpoint, rejects system-role messages (incompatible with Helix's adapter)

✓ **"qwen-flash"** — general chat-completions model, verified live with system+user message pair before wiring in

◆ **Verified live:** All 5 routing.cedar policies now correctly select their target pool end-to-end (previously only 2 of 5 were reachable). Static default fallback works unchanged.

The 5 Cedar Routing Policies

1 High-Capability Route

Complex reasoning tasks → **high_capability** pool (Qwen Max, GPT-4o, Claude)

2 Adversarial Route ✓ NOW FIXED

action_type in (bash, execute, api_call, shell) → **adversarial** pool (hardened models for untrusted inputs). Previously dead code — now fires correctly via separate action_type field.

3 Cost-Optimized Route

Simple queries, low token count → **cost_optimized** pool (lighter, faster models)

4 Sovereign Route ✓ qwen-flash verified

EU/locale-specific requirements, data residency → **sovereign** pool (qwen-flash for intl deployments)

5 Static Default Fallback

No context match → static **action_map** fallback by action field (analyze/search/write_file/summarize). Works unchanged.

◆ All **5 policies** verified live against running deployments. End-to-end routing: request → Cedar evaluation → pool selection → model inference.

JSON Receipt & Verification

Typical Receipt JSON


```
{
  "session_id": "hsess-a3f2b1c0d9e8",
  "turn": 1,
  "model": "Qwen Max",
  "pool": "adversarial",
  "response": "[FACT] Bell's theorem...",
  "claims": [{"label": "FACT",
        "text": "Bell's theorem..."}],
  "drift_score": 0.0041,
  "drift_tier": "green",
  "cedar_status": "active",
  "hash": "e3b0c44298fc1c149...",
  "chain_hash": "sha256(hex(prev)...",
  "usage": {"prompt_tokens": 812,
       "completion_tokens": 41}
}
```


Key Fields

hash — SHA-256 over entire receipt. Any edit changes this.

chain_hash — Links to every prior turn. Tampering breaks the chain.

drift_score — Fraction of text without markers (0.0 = perfect).

drift_tier — green/yellow/red coverage indicator.

claims — Every marked statement extracted as structured JSON.

cedar_status — active / fail_closed / not_configured.

Merkle Tree Verification: Every session is backed by an append-only Merkle tree. Each turn's receipt hash becomes a leaf. Inclusion proofs are available via the Sessions UI — checkable independently without trusting the server.

05

Putting It All Together

End-to-End Integration Workflow

From reading the grammar to deploying Cedar-routed production inference — a complete 5-step workflow with all 7 drift categories monitored and 5 Cedar policies verified live.

End-to-End Integration Workflow

Step 1 — Read the Grammar: Study the Helix-TTD Constitutional Grammar to understand the constraint framework, four invariants, and marker system.

Step 2 — Install the SDK: pip install helix-adapter — works with any OpenAI-compatible API.

Step 3 — Wrap Your Model: Instantiate HelixAdapter with your model_fn. Every response now carries epistemic markers and a tamper-evident receipt.

Step 4 — Deploy Helix Foundry: For production, use the Foundry API for Cedar-routed inference (5 policies, 4 pools), session management, and Merkle-backed audit trails.

Step 5 — Monitor & Audit: Track drift scores across all **7 categories**, verify receipt chains, and review Merkle inclusion proofs for compliance.

Start Building

HELIX ECOSYSTEM · APACHE 2.0

GitHub | github.com/helixprojectai-code/helix-adapter

RFC-0004 | Cedar Model Routing

Grammar | helixaiinnovations.ca/grammar

Guide | helixaiinnovations.ca/guide

Trust cannot be proprietary. Build with Helix.